



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 07 – Lecture 20: Clustering Foundations

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

The first lecture with no answer key

Topic focus: **the objective is a choice, not a fact**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, "A density-based algorithm for discovering clusters (DBSCAN)," KDD 1996.

Quick Links

No target column

Two right answers

The five families

$T = W + B$

Internal vs external

As preprocessing

Applications

Summary

Agenda

What changes when there is no target column

There is no single right answer

The taxonomy of clustering algorithms

The statistics of cluster analysis

Internal and external validation

Clustering as a preprocessing tool

General applications of clustering

Summary

How this lecture works

Every idea appears twice

1. **By hand** — six points on graph paper. Two clusterings of them, four centroids, three sums of squares and six silhouettes, all with integer arithmetic you can check with a pen.
2. **In code** — the same quantities on real data, and then the same quantities out of `scikit-learn`, agreeing to six decimal places.

Commit before you look

Four slides ask you to predict a number or a picture before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is the identity $T = W + B$ — **total scatter splits into within plus between** — worked on six points and then verified numerically. Everything in this unit is built on it.

One seed for the lecture, three that fix the data

The randomness

One seed, everywhere: `random_state=0` and `np.random.default_rng(0)`. Every k-means start, every split, every noise draw. Every listing shows it.

The data

`random_state=170` (blobs), 7 (moons), 3 (noisier moons) are not that. They *define which points exist*: change one and you are clustering other data.

Why this matters to you

Run the code as written and you get the numbers on these slides. Pin **both** `n_init` and `random_state` on `KMeans` — the default `n_init` has changed between scikit-learn versions.

Where we are: Unit VII opens, and the rules change

Everything so far had an answer key

Regression had y . Classification had a class column. Every decision — which model, which depth, which k — was settled by scoring predictions against a column that was already in the file. Cross-validation, ROC, AUC: all of it needs that column.

Today there is no such column

The data arrives as n rows and d features. *Nothing else.*

So two things must be rebuilt

1. **What is a good answer?** There is no target to be close to, so the objective has to be invented.
2. **How would we know?** There is no held-out score, so validation has to be rebuilt from the geometry alone.

Supervised and unsupervised, side by side

	Supervised (Units V–VI)	Unsupervised (Unit VII)
Input	$(x_i, y_i), i = 1 \dots n$	x_i only
Goal	predict y for a new x	find structure among the x_i
Objective	given (squared error, log loss)	chosen by you
Correct answer	exists, in the file	does not exist
Validation	held-out score	internal indices, then judgement
Failure looks like	high test error	a partition nobody can use

Clustering, defined

Partition n objects into k groups so that objects in the same group are **similar** and objects in different groups are **dissimilar**.

Every word in that sentence is a decision

What counts as similar? How many groups?
Must every object belong to exactly one?
Must groups be convex? **Change any of them and you change the answer.**

The notation for the rest of the unit

A **clustering** of $D = \{x_1, \dots, x_n\}$, $x_i \in \mathbb{R}^d$, into k groups is a map $C : D \rightarrow \{1, \dots, k\}$. Write $C_j = \{x_i : C(x_i) = j\}$ and $n_j = |C_j|$.

Hard, exhaustive, exclusive

$$\bigcup_{j=1}^k C_j = D, \quad C_a \cap C_b = \emptyset \quad (a \neq b).$$

Every object in exactly one cluster. This is what k-means and agglomerative clustering produce.

And the variants you will meet

Soft — each object gets k membership weights summing to one (Gaussian mixtures). **Partial** — some objects belong to nothing, i.e. *noise* (DBSCAN).

Hierarchical — a whole nested family, one clustering per level.

The number of ways to partition n objects into k non-empty groups is the Stirling number $S(n, k)$. For $n = 25$, $k = 3$ that is already 141 197 991 025. **Nothing in this unit searches exhaustively.**

Predict first #1 — how many clusters are in iris?

Commit to an answer

`load_iris`: 150 flowers, four measurements, and I have thrown the species column away. I run k-means for $k = 2$ and for $k = 3$ and score both with the **silhouette**, an unsupervised measure of how well separated the clusters are — higher is better.

Which k wins? And does that match the three species?

Predict first #1 — how many clusters are in iris?

Commit to an answer

`load_iris`: 150 flowers, four measurements, and I have thrown the species column away. I run k-means for $k = 2$ and for $k = 3$ and score both with the **silhouette**, an unsupervised measure of how well separated the clusters are — higher is better.

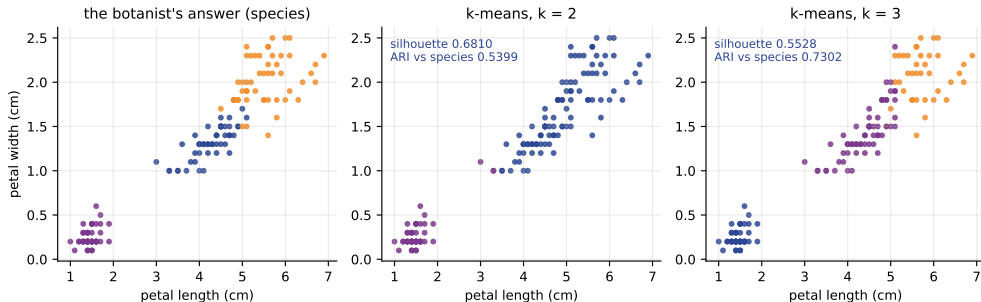
Which k wins? And does that match the three species?

Silhouette says $k = 2$ (0.6810 against 0.5528).

The species say $k = 3$ (ARI 0.7302 against 0.5399).

Both are defensible, and they disagree. Neither is a mistake.

Drawn: one dataset, two answers



Left: what a botanist calls the truth. Middle: what the geometry calls the truth. *Setosa* sits far away from the other two, and *versicolor* and *virginica* touch — so a measure that rewards separation merges them and a measure that knows the species does not.

Where the disagreement lives

```
two = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(X)
three = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
```

	k = 2	k = 3
silhouette	0.6810	0.5528
Calinski-Har.	513.92	561.63
Davies-Bould.	0.4043	0.6620
ARI vs labels	0.5399	0.7302

agreement between the two clusterings: ARI 0.5516

k = 2			k = 3		
setosa	50	0	setosa	0	50
versicolor	3	47	versicolor	48	0
virginica	0	50	virginica	14	36

At $k = 2$ the second cluster is *versicolor* and *virginica* stuck together. At $k = 3$ they are split, but imperfectly: 14 virginica land with the versicolor. **Three internal indices, and they do not agree with each other either.**

The same thing again: scaling changes the answer

A second, sharper version. `load_wine`, three known cultivars, k-means with $k = 3$ — run once on the raw table and once after standardising every column.

```
raw = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(Xw)
std = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(
    StandardScaler().fit_transform(Xw))
```

```
unscaled: silhouette in its own space 0.5711, ARI vs cultivar 0.3711
scaled   : silhouette in its own space 0.2849, ARI vs cultivar 0.8975
agreement between the two clusterings: ARI 0.3543
```

Two clusterings of the same 178 bottles that agree at ARI **0.3543** — barely more than half.
Standardising is not a preprocessing detail here; it is the definition of “similar”.

Why: one column owns the distance budget

share of the raw total variance held by each column

proline	98609.60	(99.77%)
magnesium	202.84	(0.21%)
alkalinity_of_ash	11.09	(0.01%)
color_intensity	5.34	(0.01%)

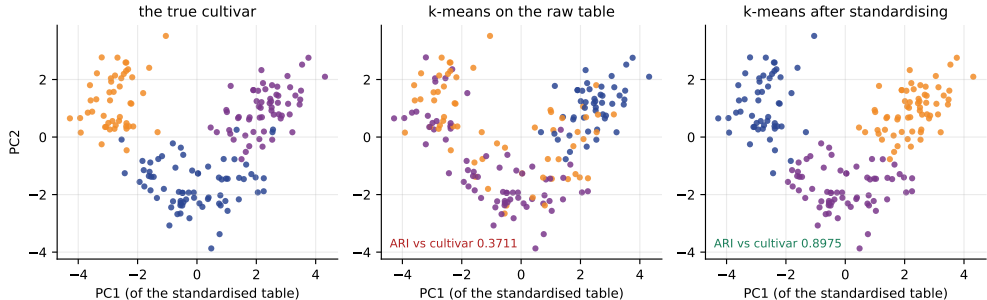
Read it

proline is measured in the hundreds; alcohol runs from 11 to 15. Euclidean distance on the raw table is 99.77% a **comparison of proline**, and the other twelve chemical measurements are decoration.

The lesson, stated once

Clustering has **no target to overrule your feature scaling**. In a supervised model a badly scaled column costs you accuracy that the test set reports. Here it silently redefines the question and nothing complains.

Drawn: the same bottles, two defensible groupings



"Clustering is in the eye of the beholder" — Estivill-Castro, 2002. Clustering is **exploratory**: it proposes a structure, it does not discover a fact. Your job is to say what you assumed, and then to say whether the structure is useful.

Five families, and what each one believes

Family	How it works	What it assumes
Partitioning	choose k centres, assign each point to the nearest, move the centres, repeat	clusters are convex blobs of similar size , one centre each
Hierarchical	merge the two closest clusters repeatedly (agglomerative) or split repeatedly (divisive)	depends entirely on the linkage : Ward assumes blobs, single linkage assumes connected chains
Density-based	grow a cluster through any region whose local point density exceeds a threshold	clusters are dense regions separated by sparse ones ; shape is free; density is roughly uniform
Grid-based	lay a grid over the space, keep the dense cells, connect the neighbours	as above, plus that a fixed resolution is adequate
Model-based	assume the data came from a mixture of distributions; fit it by maximum likelihood	the assumed family is right — e.g. ellipsoidal Gaussian components

Later lectures build hierarchical, partitioning and density-based methods in full. The point today is that **the families make different geometric commitments**, and those commitments are visible.

Predict first #2 — two interleaved crescents

Commit to a picture

Five hundred points lie on two interleaved crescent moons, each crescent a group. I run k-means with $k = 2$.

Sketch the two clusters k-means returns. Where does the boundary go?

Predict first #2 — two interleaved crescents

Commit to a picture

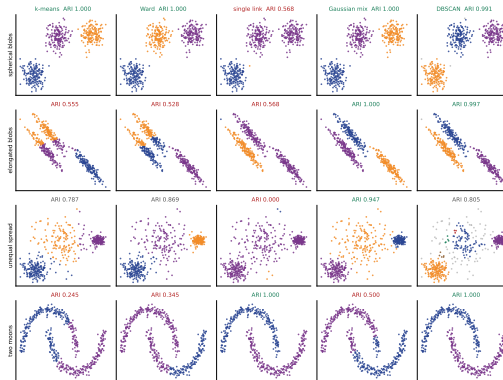
Five hundred points lie on two interleaved crescent moons, each crescent a group. I run k-means with $k = 2$.

Sketch the two clusters k-means returns. Where does the boundary go?

A straight cut, roughly vertical. ARI **0.2445**.

k-means assigns each point to the nearest of two centres, so its boundary is always the perpendicular bisector of the two centres — a **straight line**. Two crescents cannot be separated by a straight line, so it fails, and it fails *confidently*: no warning, no error, two clean-looking clusters. **DBSCAN gets ARI 1.0000**.

Drawn: four shapes, five algorithms, twenty answers



Green > 0.9 , red < 0.6 . Every column has at least one red cell.

The same thing as a table

```
for name, X, y, eps in four_shapes():
    k = len(np.unique(y))
    print(adjusted_rand_score(y, KMeans(n_clusters=k, n_init=10,
                                         random_state=0).fit_predict(X)), ...)
```

dataset (ARI)	KMeans	Ward	Single	GaussMix	DBSCAN
spherical blobs	1.0000	1.0000	0.5682	1.0000	0.9910
elongated blobs	0.5554	0.5281	0.5682	1.0000	0.9970
unequal spread	0.7872	0.8692	0.0002	0.9468	0.8054
two moons	0.2445	0.3447	1.0000	0.5003	1.0000

Read down the columns

k-means is perfect on round blobs and useless on moons. Single linkage is perfect on moons and catastrophic (0.0002) when the clusters have unequal spread — it *chains* through them.

Read across the rows

No algorithm wins every row and none is a default. You are choosing a shape assumption. Pick the family whose assumption you believe, and then check

Grid-based, built by hand in three steps

Lay a 10×10 grid over the moons. Count the points in each cell. Keep the cells with at least $\tau = 3$. Connect the survivors.

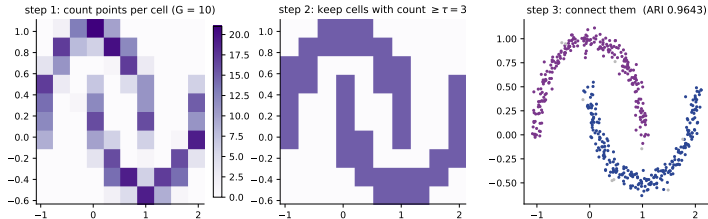
cell counts, x across, y up (a dot = fewer than tau = 3)

.	.	13	21	10	.	.	.	.	.
.	17	8	6	20	12	.	.	.	.
4	11	.	.	.	17	3	.	.	.
17	3	.	7	.	5	11	.	.	6
14	.	.	12	.	.	11	.	.	15
12	.	.	16	.	.	16	.	.	11
9	.	.	10	6	.	6	.	.	19
.	.	.	4	16	.	.	.	16	4
.	.	.	.	14	19	6	17	14	.
.	.	.	.	.	5	19	9	.	.

connected dense regions: 2

Forty-three dense cells of one hundred, falling into **two** connected regions. ARI **0.9643**, nine points left as noise — and the whole algorithm is a histogram and a flood fill.

Drawn — and the price of the grid



G=10	tau=3	dense cells	43/100	clusters	2	noise	9	ARI	0.9643
G=12	tau=2	dense cells	56/144	clusters	1	noise	9	ARI	0.0000

Loosen τ by one and the two arms **bridge into a single cluster**: ARI collapses from 0.9643 to **0.0000**. Grid methods are fast — one pass over the data, cost driven by the number of cells, not the number of points — and **brittle at the resolution**.

Three quantities, and one identity that ties them

Let $\bar{x}$ be the grand mean of all n points and c_j the **centroid** of cluster j ,

$$c_j = \frac{1}{n_j} \sum_{x \in C_j} x.$$

Total scatter

$$T = \sum_{i=1}^n \|x_i - \bar{x}\|^2$$

A property of the *data*. No clustering appears in it.

Within (WCSS)

$$W = \sum_{j=1}^k \sum_{x \in C_j} \|x - c_j\|^2$$

How tight the clusters are.
Smaller is better.

Between (BCSS)

$$B = \sum_{j=1}^k n_j \|c_j - \bar{x}\|^2$$

How far apart they sit.
Larger is better.

$$T = W + B \quad \text{— and } T \text{ does not depend on the clustering.}$$

So **minimising W and maximising B are the same problem**. That single sentence is why k-means only ever mentions W .

By hand: six points, two clusterings

$P_1(2, 2)$, $P_2(4, 2)$, $P_3(3, 5)$, $P_4(9, 3)$, $P_5(11, 3)$, $P_6(10, 6)$. Grand mean $\bar{x} = (6.5, 3.5)$.

Clustering A — left three / right three

$$c_1 = \left(\frac{2+4+3}{3}, \frac{2+2+5}{3} \right) = (3, 3)$$

$$c_2 = \left(\frac{9+11+10}{3}, \frac{3+3+6}{3} \right) = (10, 4)$$

Clustering B — low y / high y

$$c_1 = \left(\frac{2+4+9+11}{4}, \frac{2+2+3+3}{4} \right) = (6.5, 2.5)$$

$$c_2 = \left(\frac{3+10}{2}, \frac{5+6}{2} \right) = (6.5, 5.5)$$

Both are legitimate partitions into two non-empty groups. A student who only ever sees the “obvious” clustering never learns what the statistics are *for*: they are how you tell these two apart.

By hand: W , B and T for clustering A

Within, cluster by cluster

Cluster 1 about (3, 3):

$$(-1, -1): 2 \quad (1, -1): 2 \quad (0, 2): 4 \Rightarrow 8$$

Cluster 2 about (10, 4):

$$(-1, -1): 2 \quad (1, -1): 2 \quad (0, 2): 4 \Rightarrow 8$$

$$W = 8 + 8 = \mathbf{16}$$

Between, centroid by centroid

$$\|(3, 3) - (6.5, 3.5)\|^2 = 12.25 + 0.25 = 12.5$$

$$\|(10, 4) - (6.5, 3.5)\|^2 = 12.25 + 0.25 = 12.5$$

$$B = 3(12.5) + 3(12.5) = \mathbf{75}$$

Total, computed independently: $T = 22.5 + 8.5 + 14.5 + 6.5 + 20.5 + 18.5 = \mathbf{91}$, and
 $16 + 75 = 91. \checkmark$

The same thing in code — and for clustering B

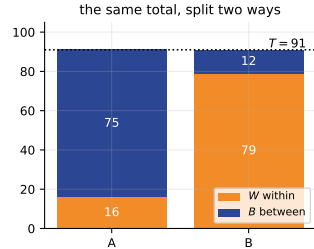
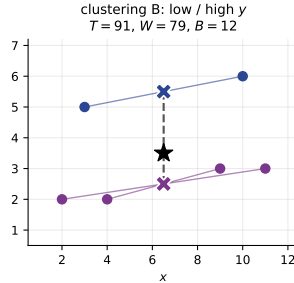
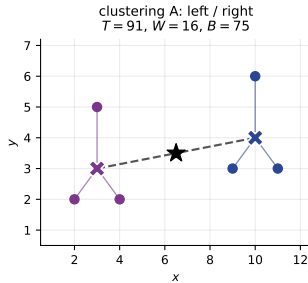
```
def scatter_decomposition(X, labels):  
    g = X.mean(axis=0)  
    total = ((X - g) ** 2).sum()  
    within = between = 0.0  
    for c in np.unique(labels):  
        M = X[labels == c]; ck = M.mean(axis=0)  
        within += ((M - ck) ** 2).sum()  
        between += len(M) * ((ck - g) ** 2).sum()  
    return total, within, between
```

clustering	T	W	B	W+B
A: left/right	91.00	16.00	75.00	91.00
B: low/high y	91.00	79.00	12.00	91.00

T does not depend on the clustering; W and B trade off inside it

Same T. Completely different split. A is tight and far apart (16 | 75); B is loose and overlapping (79 | 12). The decomposition is the instrument that says so.

Drawn: the same total, split two ways



Coloured lines are the within-cluster distances that make W ; dashed grey lines run from each centroid to the grand mean (the star) and make B . The bar on the right is the whole argument: **a fixed budget of 91, and clustering is how you divide it.**

Verified on real data, at every k

```
for k in range(1, 9):  
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)  
    T, W, B = scatter_decomposition(X, km.labels_)  
    print(k, T, W, B, abs(T - W - B))
```

k	T	W	B	T-W-B
1	681.370600	681.370600	0.000000	0.000e+00
2	681.370600	152.347952	529.022648	1.137e-13
3	681.370600	78.851441	602.519159	0.000e+00
4	681.370600	57.228473	624.142127	1.137e-13
8	681.370600	30.064593	651.306007	0.000e+00

$T = 681.370600$ at every k , and the residual never exceeds 1.14×10^{-13} — floating point, not mathematics. At $k = 1$ everything is within; as k grows the budget migrates into B .

Predict first #3 — can W choose k ?

Commit to an answer

W measures how tight the clusters are, and smaller is better. So here is the obvious procedure: try every k , compute W , keep the k with the smallest W .

What k does that procedure return, on any dataset?

Predict first #3 — can W choose k ?

Commit to an answer

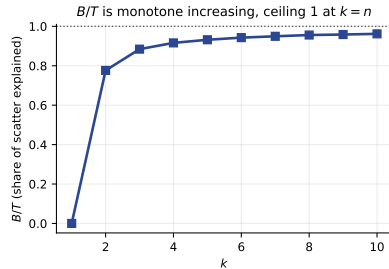
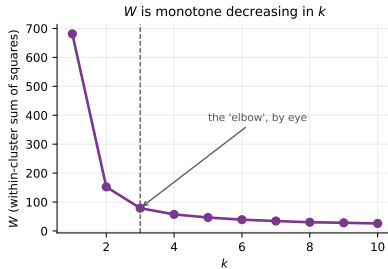
W measures how tight the clusters are, and smaller is better. So here is the obvious procedure: try every k , compute W , keep the k with the smallest W .

What k does that procedure return, on any dataset?

$k = n$. One point per cluster, and $W = 0$.

W is **monotone decreasing in k** — splitting a cluster can never increase it — so “minimise W ” is not an objective, it is a countdown. On iris: 681.37, 152.35, 78.85, 57.23, 46.45, ... all the way to zero. **Any method that chooses k must penalise k somehow.**

Drawn: monotone down, monotone up, no optimum



The “elbow” is a *visual* heuristic on a curve with no mathematical optimum, and different people read a different elbow off the same plot. It is a starting point for a conversation, not a criterion. Treat it accordingly.

Calinski–Harabasz: the decomposition, penalised

$$CH = \frac{B/(k-1)}{W/(n-k)}$$
 — between per degree of freedom, over within per degree of freedom.

By hand on the six points, clustering A: $B = 75$, $W = 16$, $k = 2$, $n = 6$, so

$CH = (75/1)/(16/4) = 75/4 = \mathbf{18.75}$. Clustering B gives $(12/1)/(79/4) = \mathbf{0.6076}$.

```
A: left/right      B/T = 0.8242    CH by hand = 18.7500    sklearn = 18.7500
B: low/high y     B/T = 0.1319    CH by hand = 0.6076    sklearn = 0.6076
k=3  B/(k-1) = 301.259579  W/(n-k) = 0.536404    ratio = 561.6278
      sklearn calinski_harabasz_score = 561.6278
```

Dividing B by $k - 1$ is what stops $k = n$ from winning: adding clusters buys you less B than it costs you in degrees of freedom. This index is nothing but the decomposition you just computed by hand.

Two kinds of validation, and only one is available

Internal — geometry only

Computed from the points and the cluster assignments. **No labels anywhere.** This is what you actually have.

- **Silhouette** $\in [-1, 1]$, higher better
- **Calinski–Harabasz**, higher better
- **Davies–Bouldin** ≥ 0 , *lower* better

External — against known groups

Requires a label column you did not use for fitting. Available on `load_iris` only because it happens to ship with one.

- **Adjusted Rand index** ≤ 1 , 0 at chance
- **Normalised mutual information** $\in [0, 1]$

In a real clustering problem you get the left column and nothing else. We use iris precisely so that we can **check the left column against the right one** — and find out where it is wrong.

By hand: the silhouette of one point

For point i in cluster C_j :

$$a(i) = \frac{1}{n_j - 1} \sum_{x \in C_j, x \neq i} d(i, x), \quad b(i) = \min_{l \neq j} \frac{1}{n_l} \sum_{x \in C_l} d(i, x), \quad s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

$P_1(2, 2)$ in clustering A

a : distances to P_2 and P_3 are 2 and

$\sqrt{10} = 3.162278$, so

$a = 5.162278/2 = 2.581139$.

b : to P_4, P_5, P_6 they are $\sqrt{50} = 7.071068$,

$\sqrt{82} = 9.055385$, $\sqrt{80} = 8.944272$, so

$b = 8.356908$.

What the three numbers mean

a is “how far am I from my own people”. b is “how far from the nearest other crowd”. s near **+1**: comfortably placed. Near **0**: on the border.

Negative: closer to another cluster than to your own — probably misclassified

All six, and the library agrees exactly

```
D = cdist(P6, P6)
```

```
s = silhouette_samples(P6, LAB_A)
```

point	a(i)	b(i)	s(i)	sklearn
P1	2.581139	8.356908	0.691137	0.691137
P2	2.581139	6.460397	0.600467	0.600467
P3	3.162278	7.213945	0.561644	0.561644
P4	2.581139	6.164881	0.581316	0.581316
P5	2.581139	8.124221	0.682291	0.682291
P6	3.162278	7.742147	0.591550	0.591550
mean silhouette	0.618068		sklearn	0.618068
clustering B		-0.071164	sklearn	-0.071164

Clustering B scores **-0.071164** — negative, meaning the average point is nearer the *other* cluster's members than its own. The silhouette does not merely prefer A; it says B is actively wrong.

Davies–Bouldin, in one line

$$\text{DB} = \frac{1}{k} \sum_{j=1}^k \max_{l \neq j} \frac{S_j + S_l}{d(c_j, c_l)}, \quad S_j = \frac{1}{n_j} \sum_{x \in C_j} \|x - c_j\|.$$

For every cluster, find its *worst* neighbour — the one whose combined spread is largest relative to the centroid distance — and average those worst cases. **Lower is better**, and 0 is unattainable.

Three indices, three formulas

Silhouette works on **pairwise distances**;
CH works on the **scatter decomposition**;
DB works on **centroid spreads**. They
measure related but different things.

So expect them to disagree

And they do, on the very next slide. An index is a summary of a partition into one number; three different summaries of the same partition need not rank two partitions the same way.

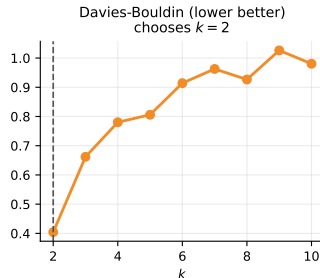
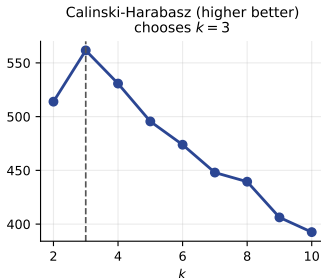
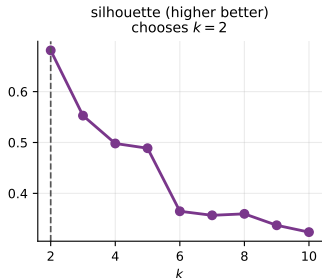
All three across k , no labels used

```
for k in range(2, 9):  
    lab = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(X)  
    print(k, silhouette_score(X, lab), calinski_harabasz_score(X, lab),  
          davies_bouldin_score(X, lab))
```

k	silhouette	CH	DB	ARI	NMI
2	0.6810	513.92	0.4043	0.5399	0.6565
3	0.5528	561.63	0.6620	0.7302	0.7582
4	0.4981	530.77	0.7803	0.6498	0.7219
5	0.4887	495.54	0.8060	0.6079	0.6939
6	0.3648	473.85	0.9142	0.4475	0.6380
7	0.3569	447.96	0.9628	0.4813	0.6689
8	0.3597	439.46	0.9266	0.4570	0.6581

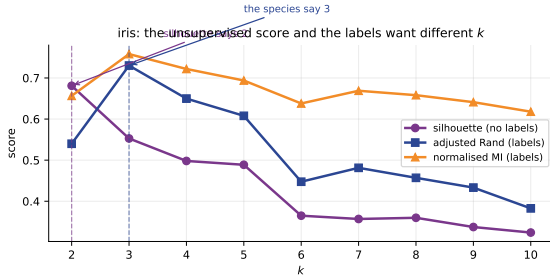
Silhouette picks $k = 2$. Calinski–Harabasz picks $k = 3$. Davies–Bouldin picks $k = 2$. **Two of the three internal indices disagree with the species**, which pick $k = 3$ by both external measures.

Drawn: the internal indices, and where each one peaks



Report the ones that disagree with you. A single index quoted alone is a number chosen after the fact to support a decision already taken.

Drawn: unsupervised score against ground truth



Be honest about this

The silhouette **prefers a clustering the labels do not**. $k = 2$ scores 0.6810 but merges two of the three species; $k = 3$ scores only 0.5528 and recovers them.

And say why

Silhouette rewards *separation*. *Versicolor* and *virginica* genuinely overlap in these four measurements, so a separation-based score is right

Predict first #4 — what does random labelling score?

Commit to a number

Take iris. Ignore the data entirely. Assign each of the 150 flowers a cluster in $\{0, 1, 2\}$ by rolling a die. Score that against the true species with the plain **Rand index** — the fraction of point pairs the two partitions agree about.

What does pure noise score?

Predict first #4 — what does random labelling score?

Commit to a number

Take iris. Ignore the data entirely. Assign each of the 150 flowers a cluster in $\{0, 1, 2\}$ by rolling a die. Score that against the true species with the plain **Rand index** — the fraction of point pairs the two partitions agree about.

What does pure noise score?

0.5567. Over half, from nothing at all.

The **adjusted** Rand index subtracts the expected value under random labelling: it scores **−0.0010**. NMI scores 0.0114. **Always report the adjusted form**, and treat any unadjusted agreement statistic near 0.5 as evidence of nothing.

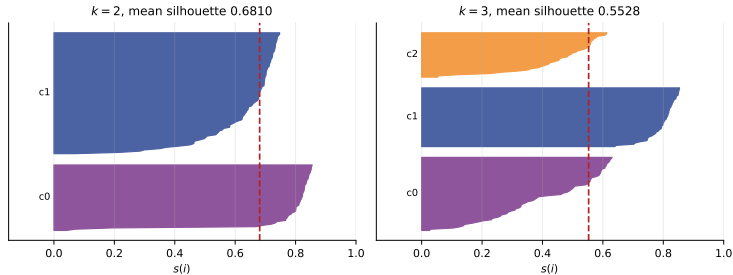
Cluster numbers are names, not classes

```
lab3 = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
perm = np.array([0: 2, 1: 0, 2: 1][v] for v in lab3) # rename only
```

```
ARI before 0.730238   after 0.730238
NMI before 0.758176   after 0.758176
200 completely random 3-way labellings of iris
Rand index              mean +0.5567   sd 0.0035
adjusted Rand index      mean -0.0010   sd 0.0073
normalised mutual info    mean +0.0114   sd 0.0067
```

Renaming the clusters changes nothing, because ARI and NMI are functions of the **partition**, not of the labels. This is also why `accuracy_score(y, clusters)` is meaningless: cluster 0 is not class 0, it is just the first group the algorithm happened to build.

Drawn: the silhouette profile, which the mean hides



At $k = 3$ the three clusters score 0.4173, **0.7981** and 0.4511: one crisp cluster (*setosa*) and two mediocre ones. The mean of 0.5528 conceals that entirely. **Plot the profile, not just the average.**

Two ways clustering feeds a supervised model

1. Cluster membership as a feature

Fit k-means on the training rows. Append the cluster indicator (or the k distances to the centroids) to X . A **linear** model then gets a piecewise, non-linear view of the space for the price of k extra columns.

2. Cluster-then-label

Labels cost money; rows are free. Cluster the unlabelled rows, label **one representative per cluster**, and either train on those few or propagate each label to its whole cluster. This is how a labelling budget should be spent.

And the rule that governs both

Fit the clusterer on the **training rows only**, then predict on the test rows. Fitting k-means on all the data before splitting leaks the test set into the features — the same leakage rule as scaling, wearing a new hat.

Measured: cluster indicators on the moons

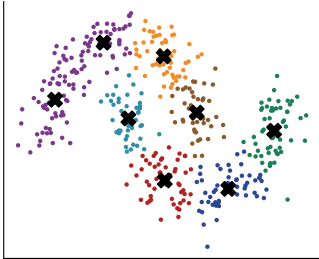
```
km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(Xtr)
Otr = np.eye(k)[km.labels_]; Ote = np.eye(k)[km.predict(Xte)]
LogisticRegression(max_iter=5000).fit(np.hstack([Xtr, Otr]), ytr)
```

```
logistic regression on the two raw columns    0.9056
k      + one-hot      + distances
2      0.9444          0.9667
4      0.9722          0.9167
6      0.9889          0.9778
8      0.9944          0.9833
10     0.9944          0.9833
15     0.9944          0.9889
20     0.9889          0.9889
```

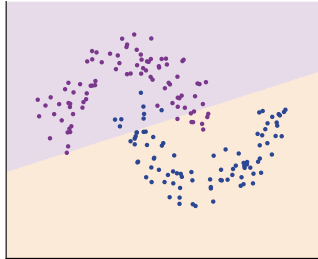
0.9056 → **0.9944** — a linear model that could not bend now bends, because the eight cluster indicators carve the plane into eight regions and the model fits an intercept in each. **The labels were never used to build those regions.**

Drawn: where the extra columns come from

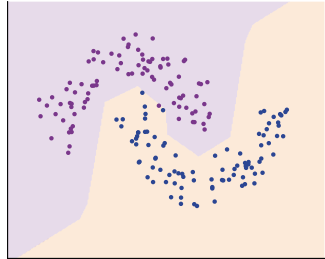
step 1: k-means, $k = 8$, labels unused



logistic regression on x_1, x_2
test 0.9056



and on the 8 cluster indicators
test 0.9944



Left: k-means run on the training rows with the labels hidden. Middle: logistic regression on two columns can only draw a line. Right: the same model, plus eight indicator columns, now draws a boundary that follows the cluster edges.

And the null result, reported as it happened

logistic regression on the 30 scaled columns	0.9591
+ 2 cluster distances	0.9532
+ 5 cluster distances	0.9591
+ 10 cluster distances	0.9649
+ 20 cluster distances	0.9591

On breast_cancer, it buys nothing

Best case +0.0058, or **one test patient out of 171**. Two of the four settings made it worse. This is not a result; it is noise.

Why the moons and not this

The trick pays when the **decision boundary is non-linear and the clusters line up with it**. Here thirty scaled columns are already close to linearly separable, so there is nothing for the extra columns to add. **Measure it; do not assume it.**

Cluster-then-label: spending fifty labels well

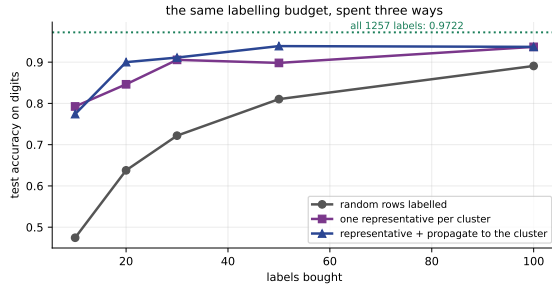
```
km = KMeans(n_clusters=B, n_init=10, random_state=0).fit(Xtr)
rep = np.argmax(km.transform(Xtr), axis=0) # one nearest-to-centroid row
yprop = np.empty(len(Xtr), int)
for i in range(B): yprop[km.labels_ == i] = ytr[rep[i]]
```

fully supervised baseline (1257 labels) 0.9722

budget	random	sd	representative	propagated
10	0.4744	0.0180	0.7926	0.7741
20	0.6378	0.0337	0.8463	0.9000
30	0.7219	0.0374	0.9056	0.9111
50	0.8104	0.0274	0.8981	0.9389
100	0.8907	0.0184	0.9370	0.9370

Fifty labels chosen *by clustering*, then propagated, reach **0.9389** — against 0.8104 for fifty labels chosen at random, and 0.9722 for all 1257. **Ninety-six per cent of the achievable accuracy for four per cent of the labelling.**

Drawn: three ways to spend the same budget



50 labels bought; 1257 rows
labelled by propagation
correct: 1206 of 1257 (0.9594)
trains on 4.06% wrong labels

Why propagation still wins

It introduces 51 wrong labels. It also multiplies the training set by 25. Logistic regression tolerates 4% label noise far better than it tolerates having only fifty rows.

Where this actually gets used

Segmentation

Customers by purchase behaviour, so that a campaign can be written for each group rather than for an average nobody resembles. **Students** by attempt patterns, to decide who needs which intervention.

Organising a corpus

Documents into topics for browsing; **search results** grouped so a query with two meanings returns two piles; **genes** with correlated expression profiles.

Compression and summarisation

Vector quantisation: replace every value by its cluster centre. Colour palettes, codebooks, and the first step of many retrieval systems.

Anomaly detection

Points in no cluster, or in a cluster of size one, are candidates: **fraudulent transactions, intrusions, faulty sensors**. DBSCAN gives you this for free — it already refuses to assign some points.

Measured: colour quantisation is clustering

Treat each pixel as a point in $\mathbb{R}^3$. Cluster the colours. Replace every pixel by its cluster centre.

image 427 x 640 pixels, 96615 distinct colours

k= 2	palette	2 colours	MSE 0.019764	storage 4.17% of 24-bit
k= 4	palette	4 colours	MSE 0.007028	storage 8.34% of 24-bit
k= 8	palette	8 colours	MSE 0.003246	storage 12.51% of 24-bit
k= 16	palette	16 colours	MSE 0.001792	storage 16.69% of 24-bit
k= 32	palette	32 colours	MSE 0.000998	storage 20.88% of 24-bit

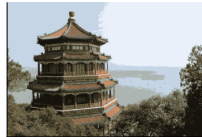
original: 96615 colours



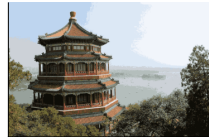
k = 2 MSE 0.019764



k = 8 MSE 0.003246



k = 16 MSE 0.001792



Measured: digits, with the labels hidden

```
k-means with k=10 on 1797 unlabelled digit images
ARI vs the true digit 0.6657, NMI 0.7425
cluster : dominant digit (purity)
  0 : 0 (176 of 178 = 0.99)    5 : 6 (177 of 182 = 0.97)
  1 : 1 (100 of 223 = 0.45)    6 : 4 (165 of 169 = 0.98)
  2 : 7 (174 of 208 = 0.84)    7 : 5 (136 of 150 = 0.91)
  3 : 1 ( 54 of  87 = 0.62)    8 : 9 (139 of 247 = 0.56)
  4 : 3 (154 of 178 = 0.87)    9 : 2 (148 of 175 = 0.85)
overall purity 0.7919
```

Knowing nothing but 64 pixel values, k-means finds groups that are 79% pure. Zeros, sixes and fours come out almost perfectly. The failures are honest ones: **ones are split across two clusters**, and nines and eights are confused — which is roughly what a human confuses too.

Summary — twelve lines

1. Clustering has **no target column**, so the objective is a modelling choice and validation must be rebuilt from geometry.
2. **There is no single right answer.** On iris the silhouette prefers $k = 2$ (0.6810 against 0.5528) and the species prefer $k = 3$ (ARI 0.7302 against 0.5399). The two clusterings agree at ARI **0.5516**.
3. Feature scaling is the definition of similar. On wine, raw against standardised gives ARI to the cultivar of 0.3711 and **0.8975**, and the two clusterings agree at only 0.3543 — because proline holds 99.77% of the raw variance.
4. Five families: **partitioning, hierarchical, density-based, grid-based, model-based**, each committing to a different cluster shape.
5. Those commitments have teeth. On two moons k-means scores **0.2445** and DBSCAN **1.0000**; on unequal spread single linkage scores **0.0002** and the Gaussian mixture 0.9468. **No algorithm wins all four datasets.**
6. A grid clusterer is a histogram plus a flood fill: 43 dense cells of 100, two components, ARI 0.9643 — and at $\tau = 2$ the arms bridge and it collapses to **0.0000**.
7. $T = W + B$. On six points: $91 = 16 + 75$ for one clustering and $91 = 79 + 12$ for another. On iris, $T = 681.370600$ at every k , residual $\leq 1.14e-13$.
8. T is fixed, so **minimising W and maximising B are one problem**. W is monotone decreasing in k and reaches 0 at $k = n$, so it cannot choose k ; the elbow is a heuristic, not an optimum.
9. $CH = \frac{B/(k-1)}{W/(n-k)}$ is the decomposition with the degrees of freedom put back: $75/4 = 18.75$ by hand, 18.7500 from scikit-learn.
10. Silhouette: $s(i) = (b - a) / \max(a, b)$. By hand P_1 gives **0.691137**; the mean over the six is 0.618068 and the rival clustering is **-0.071164** — negative means misassigned.
11. External measures must be **chance-corrected**: a random 3-way labelling of iris scores Rand **0.5567** but ARI **-0.0010**. Renaming clusters changes neither ARI nor NMI, so “accuracy” is undefined.
12. As preprocessing it works when it works: moons 0.9056 $\rightarrow$ **0.9944** with eight cluster indicators; breast_cancer 0.9591 $\rightarrow$ 0.9649, which is one patient. Cluster-then-label reaches **0.9389** on 50 labels against 0.8104 spent at random.

Before the next lecture

Read

notes.pdf — the decomposition proved, the silhouette derived, the taxonomy in full, and **practice questions with complete worked solutions**.

Do

Take the six points, invent a *third* clustering, and compute W , B , T , CH and the mean silhouette with a pen. Check that T still comes to 91. If it does not, one of your centroids is wrong.

The habit to take away

Before you cluster anything, write down **what similar means** and **what you will do with the groups**. Both answers change the algorithm you should pick, and neither is in the data file.

And the habit to unlearn

Do not report one silhouette and call the clustering validated. Report at least two indices, plot the profile, look at the clusters, and say plainly which measures disagreed with you.

Next: what “similar” means for binary, nominal and ordinal columns — the distances this lecture kept assuming. Materials:

<https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.